

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 02 | Complexity Analysis: Measuring Algorithm Efficiency (Big-O)

Week 2 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 02 at a glance

Item	Details
Lab title	Complexity Analysis: Measuring Algorithm Efficiency (Big-O)
Week	2 of 16
Prerequisites	Lab 01 (Python basics, loops, functions, lists, dictionaries)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab02_Instructor.ipynb , Lab02_Student.ipynb
Next lab	Lab 03: Arrays and Python Lists

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain why we measure algorithms by **number of steps** and not only by seconds.
2. Count the basic operations of simple code and express growth with **Big-O notation**.
3. Recognise the common complexity classes: **$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$** .
4. Analyse **best, average and worst case** of an algorithm.
5. Calculate the **space complexity** of a program.
6. Measure running time experimentally in Python and compare it with theory.
7. Choose the more efficient of two solutions for the same problem.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain time and space complexity and asymptotic notations	Understand (C2)
CLO-3	Analyse the efficiency of algorithms using Big-O	Analyse (C4)

2. Motivation: Why Does Efficiency Matter?

In Lab 01 you saw that the dictionary version of Two Sum was hundreds of times faster than the nested-loop version, even though both gave the same answer. Today you will learn the **language** professionals use to describe that difference: **complexity analysis** and **Big-O notation**.



A real story: the result website on result day

Every year when board or university results are announced, some result websites crash. Lakhs of students search for their roll number at the same time. If the website searches roll numbers one by one ($O(n)$), every request takes long and the server collapses. If it uses an index ($O(\log n)$) or a hash table ($O(1)$), it survives easily.

Same hardware, different algorithm, completely different outcome.

2.1 Complexity in Daily Life

Situation	Slow way	Fast way
Finding a word in a dictionary book	Read every page from the start: $O(n)$	Open the middle, go left or right: $O(\log n)$
Finding a contact in your phone	Scroll through all contacts: $O(n)$	Type the name, search index: $O(\log n)$ or $O(1)$
Checking attendance of 60 students	For each name, scan the whole list: $O(n^2)$	Mark roll numbers in a set: $O(n)$
Getting the first item in a queue	-	Always one step: $O(1)$

2.2 Complexity in Industry

- Every technical interview ends with the question: **“What is the time and space complexity of your solution?”** If you cannot answer, your solution is considered incomplete.
- Cloud companies (AWS, Azure, Google Cloud) charge by computing time. An $O(n^2)$ algorithm on big data can cost thousands of dollars more than an $O(n \log n)$ one.
- Mobile apps must be efficient to save battery and memory on low-end phones, which are very common in Pakistan.

2.3 Complexity in Research

- **AI:** the Transformer model behind ChatGPT has attention with $O(n^2)$ cost in sentence length. Much modern research (e.g., linear attention, FlashAttention) is about reducing this complexity.
- **Cryptography:** your bank's security depends on problems that take exponential time to break.
- **P vs NP**, the most famous open problem in computer science, is entirely about complexity classes.



2.4 Why It Matters in 2026–2030



Data keeps growing, so efficiency matters more every year

When n grows from 1,000 to 1,000,000,000, an $O(n)$ algorithm becomes 1 million times slower, but an $O(n^2)$ algorithm becomes **1 trillion times** slower.

As data grows with AI and IoT, choosing the right complexity class is the difference between software that works and software that never finishes.

AI tools often produce working but inefficient code. Your ability to say “this is $O(n^2)$, we can do $O(n)$ ” is a skill that keeps you ahead.



3. Part A: How Do We Measure an Algorithm?

3.1 Why Not Just Use a Stopwatch?

We could time a program in seconds, but the result depends on many things that have nothing to do with the algorithm:

- The speed of the computer (a new laptop vs an old lab PC).
- The programming language (C++ is usually faster than Python).
- Other programs running at the same time.
- The size and type of input.

So instead, we count **how many basic steps** the algorithm performs as a function of the **input size n** . This is machine-independent. A basic step is a simple operation: an assignment, a comparison, an arithmetic operation or accessing a list element.

3.2 Counting Steps: Worked Example

```
def total_marks(marks):          # n = len(marks)
    total = 0                    # 1 step
    for m in marks:              # loop runs n times
        total = total + m        # 1 step each time -> n steps
    return total                 # 1 step
# Total steps ≈ n + 2
print(total_marks([10, 20, 30]))
```

The exact count is $n + 2$. When n is very large (say 1 crore), the “+2” does not matter at all. What matters is that the work **grows in direct proportion to n** . We say this algorithm is **$O(n)$** , read as “order n ” or “Big-O of n ”.

3.3 Rules for Finding Big-O

Rule	Meaning	Example
1. Drop constants	Constant factors do not change the growth	$3n + 5 \rightarrow O(n)$
2. Keep the biggest term	For large n , the largest term dominates	$n^2 + 100n + 7 \rightarrow O(n^2)$
3. Sequential parts add	Two loops one after another	$O(n) + O(n) = O(n)$
4. Nested parts multiply	A loop inside a loop	$O(n) \times O(n) = O(n^2)$
5. Different inputs, different letters	Two lists of sizes a and b	Nested over both $\rightarrow O(a \times b)$



4. Part B: Common Complexity Classes

Big-O	Name	Example	$n = 1,000 \rightarrow$ steps
$O(1)$	Constant	Access <code>marks[5]</code> , dictionary lookup	1
$O(\log n)$	Logarithmic	Binary search (Lab 04)	≈ 10
$O(n)$	Linear	Find max, linear search	1,000
$O(n \log n)$	Linearithmic	Merge sort, quick sort (Lab 07)	$\approx 10,000$
$O(n^2)$	Quadratic	Bubble sort, checking all pairs	1,000,000
$O(2^n)$	Exponential	Naive recursive Fibonacci (Lab 06)	$\approx 10^{300}$ (never finishes)



Remember the order

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Everything up to $O(n \log n)$ is considered efficient for large data. $O(n^2)$ is fine only for small inputs. Exponential is practical only for tiny inputs.

4.1 O(1): Constant Time

```
def get_first(items):
    return items[0]          # one step, whether the list has 5 or 5 crore
                             items

prices = {"sugar": 160, "rice": 320}
print(get_first([4, 8, 15]), prices["rice"])  # dictionary lookup is O(1) on
average
```

4.2 O(n): Linear Time

```
def contains(items, target):
    for x in items:          # in the worst case we check all n items
        if x == target:
            return True
    return False

print(contains([5, 3, 9, 1], 9))
```

4.3 O(n²): Quadratic Time

```
def has_duplicate_slow(items):
    n = len(items)
    for i in range(n):      # n times
        for j in range(i + 1, n):  # up to n times
            if items[i] == items[j]:
                return True
    return False

print(has_duplicate_slow([3, 1, 4, 1, 5]))
```

The inner loop runs $(n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2}$ times. After dropping constants and smaller terms, that is **O(n²)**.

4.4 O(log n): Logarithmic Time

An algorithm is O(log n) when it **cuts the problem in half** at every step. Starting with $n = 1,000,000$, halving gives $500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$ in only about **20 steps**, because $2^{20} \approx 1,000,000$.

```
def halving_steps(n):
    steps = 0
    while n > 1:
        n = n // 2          # problem size is cut in half
        steps += 1
    return steps

for n in [8, 1_000, 1_000_000, 1_000_000_000]:
    print(f"n = {n:>13}, -> {halving_steps(n)} steps")
```

4.5 Seeing the Growth

Run this cell to print a growth table. Notice how fast $O(n^2)$ and $O(2^n)$ explode compared to the others.

```
import math

print(f"{'n':>6} {'log n':>7} {'n log n':>9} {'n^2':>10} {'2^n':>22}")
for n in [1, 10, 20, 50, 100, 1000]:
    two_n = 2 ** n if n <= 60 else float("inf")
    print(f"{'n':>6} {'math.log2(n)':>7.1f} {'n * math.log2(n)':>9.0f} {'n * n':>10} {'two_n':>22}")
```



5. Part C: Best, Average and Worst Case

The same algorithm can take a different number of steps on different inputs of the same size. Consider searching for a roll number in a list of n students:

Case	When it happens	Steps	Big-O
Best case	The target is the first element	1	$O(1)$
Average case	The target is somewhere in the middle	$\approx n/2$	$O(n)$
Worst case	The target is last or not present	n	$O(n)$

In practice we usually report the **worst case**, because it gives a **guarantee**: the algorithm will never be slower than this. When an engineer says “linear search is $O(n)$ ”, they mean the worst case.



Other notations you will meet

- **Big-O (O):** upper bound, “at most this much” (used most often).
- **Big-Omega (Ω):** lower bound, “at least this much”.
- **Big-Theta (Θ):** tight bound, both upper and lower.

In interviews and in this course, Big-O for the worst case is enough unless your theory course says otherwise.



6. Part D: Space Complexity

Space complexity is the extra memory an algorithm needs as the input grows. We usually count **auxiliary space**: memory used in addition to the input itself.

```
def reverse_new(items):          # O(n) extra space: builds a new list
    result = []
    for x in items:
        result.insert(0, x)
    return result

def reverse_in_place(items):     # O(1) extra space: only two index
    variables
    left, right = 0, len(items) - 1
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1
    return items

print(reverse_new([1, 2, 3, 4]), reverse_in_place([1, 2, 3, 4]))
```



The time–space trade-off

Often you can make a program **faster by using more memory**. Two Sum with a dictionary uses $O(n)$ extra memory to get $O(n)$ time instead of $O(n^2)$. Engineers choose based on which resource is more precious.

Can you spot a hidden cost? `result.insert(0, x)` shifts every element, so `reverse_new` is actually **$O(n^2)$ time**. You will understand why in Lab 03.



7. Part E: Measuring Time in Python

Theory predicts growth; experiments confirm it. Python’s `time.perf_counter()` gives a precise clock.

```

import time

def measure(func, data):
    start = time.perf_counter()
    func(data)
    return time.perf_counter() - start

def linear_work(data):
    total = 0
    for x in data:
        total += x
    return total

def quadratic_work(data):
    count = 0
    for x in data:
        for y in data:
            count += 1
    return count

print(f"{'n':>6} {'O(n) seconds':>14} {'O(n^2) seconds':>16}")
for n in [250, 500, 1000, 2000]:
    data = list(range(n))
    print(f"{'n':>6} {measure(linear_work, data):>14.6f} {measure(quadratic_work, data):>16.6f}")

```

When n **doubles**, the $O(n)$ time roughly doubles, but the $O(n^2)$ time roughly becomes **four times** larger. That is the signature of quadratic growth.

8. C++ Corner

Complexity is language-independent: the same loops have the same Big-O in C++. C++ just runs each step faster.

C++

```

#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using namespace std::chrono;

long long quadraticWork(const vector<int>& data) {
    long long count = 0;
    for (int x : data)
        for (int y : data)
            count++;
    return count;
}

int main() {
    for (int n : {1000, 2000, 4000}) {
        vector<int> data(n, 1);
        auto start = high_resolution_clock::now();
        quadraticWork(data);
        auto ms = duration_cast<microseconds>(high_resolution_clock::now() -
start).count();
        cout << "n = " << n << " -> " << ms << " microseconds" << endl;
    }
}

```

9. Complexity Cheat Sheet

Code pattern	Big-O
Simple statements, no loops	$O(1)$
One loop from 0 to n	$O(n)$
Loop where i doubles or halves each time	$O(\log n)$
Two nested loops over n	$O(n^2)$
Loop over n, inside a halving loop	$O(n \log n)$
Two separate loops over n and m	$O(n + m)$
<code>x in list</code>	$O(n)$
<code>x in set, dict[key]</code>	$O(1)$ average
<code>list.append(x), list.pop()</code>	$O(1)$ amortised
<code>list.insert(0, x), list.pop(0)</code>	$O(n)$
<code>sorted(list)</code>	$O(n \log n)$

10. Common Mistakes

Mistake	Why it is wrong	Correct thinking
“My code ran in 0.01 s, so it is efficient”	Small inputs hide bad growth	Test with large n and analyse Big-O
Writing $O(2n)$ or $O(n/2)$	Constants are dropped	Both are $O(n)$
Ignoring built-in costs	x in list inside a loop is $O(n)$ each time	That loop becomes $O(n^2)$; use a set
Nested loops are always $O(n^2)$	Inner loop may run a fixed number of times	Loop of $n \times$ loop of 5 = $O(n)$
Forgetting space	A new list of size n uses $O(n)$ memory	Always state time and space

11. Lab Tasks

Solve all tasks in `Lab02_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Count the Steps

Write `count_steps(n)` that returns how many times the line `count += 1` runs in the code below, **without running the loops** (use a formula). Then state its Big-O in a comment.

`for i in range(n): / for j in range(n): / count += 1` followed by a separate loop `for k in range(n): count += 1.`

```
count_steps(10) → 110      # n*n + n
count_steps(0)  → 0
```

Task 2 Identify the Big-O

For each function in the notebook, fill the dictionary `answers` with its Big-O as a string, choosing from `"O(1)"`, `"O(log n)"`, `"O(n)"`, `"O(n log n)"`, `"O(n^2)"`.

Task 3 Contains Duplicate: $O(n^2) \rightarrow O(n)$

Write `has_duplicate_fast(items)` that returns `True` if any value appears at least twice. It must be $O(n)$ time using a `set`. Compare it with `has_duplicate_slow` from the manual.

In a comment, state the time and space complexity of both versions.

```
[3, 1, 4, 1, 5] → True
[1, 2, 3]       → False
```

Task 4 Missing Roll Number

A class has roll numbers 1 to n , but one student's roll number is missing from the attendance list (the list has $n-1$ numbers, in any order). Write `find_missing(rolls, n)` in $O(n)$ time and $O(1)$ extra space.

Hint: the sum of 1 to n is $n(n+1)/2$.

```
find_missing([3, 1, 5, 2], 5) → 4
```

Task 5 Prove the Growth Experimentally

Write `doubling_ratio(func, n)` that measures `func(list(range(n)))` and `func(list(range(2 * n)))` and returns the ratio `time(2n) / time(n)`.

Use it on `linear_work` and `quadratic_work` from the manual. A ratio near 2 suggests $O(n)$; near 4 suggests $O(n^2)$. Explain your results in a markdown cell.

Task 6 Fast Power: $O(n) \rightarrow O(\log n)$

Computing x^n by multiplying x by itself n times is $O(n)$. Write `fast_power(x, n)` that is **$O(\log n)$** using this idea: if n is even, $x^n = (x^2)^{(n/2)}$; if n is odd, $x^n = x \cdot x^{n-1}$. Do not use `**` or `pow()`.

Also return the number of multiplications performed, so you can see it grows like $\log n$.

```
fast_power(2, 10) → (1024, multiplications)    # around 5–6, not 10
```

Task 7 Common Elements of Two Lists

Two sections submitted lists of students who joined a society. Write `common_students(a, b)` that returns a **sorted list** of names present in both lists, without duplicates.

Write it once with nested loops ($O(a \times b)$) and once with sets ($O(a + b)$) as `common_students_fast`. State both complexities.

```
a = ["Ali", "Sara", "Hamza", "Ali"], b = ["Sara", "Usman", "Ali"]
→ ["Ali", "Sara"]
```

12. LeetCode Practice

Solve these on leetcode.com and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab02` folder.

#	Problem	Level	Concept practised
217	Contains Duplicate	Easy	Set, $O(n)$ vs $O(n^2)$
268	Missing Number	Easy	Math / $O(1)$ space
1342	Number of Steps to Reduce a Number to Zero	Easy	Halving $\rightarrow O(\log n)$
1365	How Many Numbers Are Smaller Than the Current Number	Easy	Brute force vs counting

#	Problem	Level	Concept practised
1295	Find Numbers with Even Number of Digits	Easy	Analysing digit loops
50	Pow(x, n)	Medium	Fast power, $O(\log n)$



13. Home Task

1. Complete any unfinished lab tasks and push the notebook to [DSA-Labs/Lab02](#).
2. For every function you wrote in Lab 01, write its time and space complexity as a comment.
3. Write a one-page note (in a markdown cell) comparing $O(n)$ and $O(n^2)$ with your own real-life example from Pakistan (e.g., NADRA, results, cricket scores).
4. **Mini-project:** build a small “Algorithm Race” in the notebook that times linear search vs set lookup for $n = 10^3, 10^4, 10^5, 10^6$ and prints a neat table.
5. Solve at least 2 LeetCode problems from the list and write the complexity of each solution.

? 14. Viva Questions

1. Why do we count steps instead of measuring seconds?
2. What does $O(n)$ mean in simple words?
3. Why do we drop constants in Big-O? Give an example.
4. Arrange in increasing order: $O(n^2)$, $O(1)$, $O(n \log n)$, $O(\log n)$, $O(n)$.
5. What is the difference between best, average and worst case?
6. Why is a halving loop $O(\log n)$?
7. What is space complexity? What is the space complexity of reversing a list in place?
8. Explain the time–space trade-off with Two Sum.
9. What is the complexity of `x in list` and `x in set`?
10. If n doubles, how does the running time of an $O(n^2)$ algorithm change?



15. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



16. References and Further Learning

- Big-O Cheat Sheet: bigocheatsheet.com
- Python TimeComplexity wiki: wiki.python.org/moin/TimeComplexity
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.